

Project: Local Multimodal Hybrid RAG Agent

Class: CSCI E-89, Deep Learning, Harvard Extension School, Fall '25

Professors: Prof. Zoran B. Djordjević & Prof. Rahul Joglekar

Student: Robert Frenette

Problem Statement

Traditional chatbots face significant limitations when handling complex, domain-specific queries that require access to both structured data (DBs) and unstructured documents (DOCs, PDFs). Standard LLMs suffer from hallucinations, lack access to real-time data, and cannot seamlessly integrate multiple knowledge sources. Also, cloud-based AI services pose critical security and compliance risks for industries handling sensitive information (financial services, healthcare, insurance) where customer data, proprietary policies, and confidential documents cannot be transmitted over the public internet due to regulatory requirements (HIPAA, PCI) and data governance concerns.

This project demonstrates a hybrid retrieval augmented generation (RAG) Agent that intelligently routes user queries between a SQL database (SQLite) and unstructured document retrieval (ChromaDB vector store), providing source-grounded responses while maintaining complete data privacy through local (Ollama) processing.

Dataset

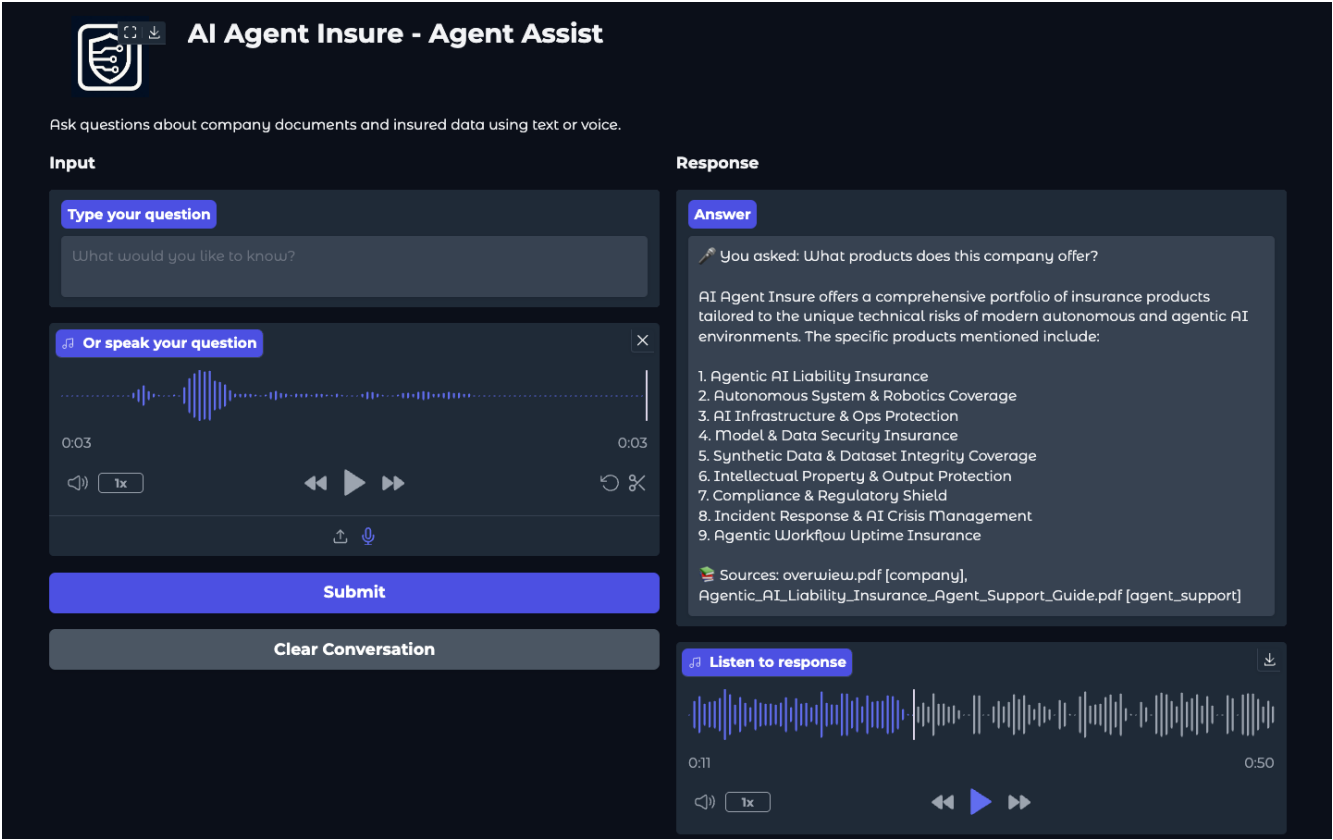
This project utilizes two complementary datasets for a fictitious insurance company (AI Agent Insure). The first is a structured SQLite DB containing 100 insureds with 208 policies, 46 claims, and associated coverage details, risk profiles, and claims history in five related tables. The other is an unstructured knowledge base of 20+ PDF documents organized into categories such as company information, agent support guides, procedural documentation, and personnel information. These PDFs are processed into vector embeddings using nomic-embed-text, then put into a ChromaDB vector store with enhanced metadata (document type, chunk indices, and page counts).

Technology and Features Demonstrated

The application showcases a hybrid RAG architecture built with LangChain, running entirely on local infrastructure using Ollama for LLM inference (llama3.2:3b) and embeddings (nomic-embed-text).

Key features include: Intelligent query routing that automatically detects whether queries require SQL database operations or RAG document retrieval; a complete RAG pipeline with optimized retrieval (top-k=4), custom prompt engineering to minimize hallucinations, and summary-based conversational memory for multi-turn dialogues; multimodal interaction supporting both text and voice input (OpenAI Whisper) with audio output (text-to-speech); and enhanced source attribution with document metadata, enabling users to verify information origins.

The *AI Agent Insure – Agent Assist App* is a local, multimodal RAG Agent that runs on Ollama. It has a Gradio UI with which the User can submit queries about the company in text or speech, and get a text and audio response.



Note: *AI Agent Insure* is a fictitious company created for this project.

Technology Stack

The application is built using a local-first tech stack which runs on Ollama:

Framework: LangChain for RAG pipeline orchestration

LLM: llama3.2:3b via Ollama for local inference

Embeddings: nomic-embed-text via Ollama

Vector Database: ChromaDB for persistent vector storage

SQL Database: SQLite3 (built-in Python library)

UI Framework: Gradio for web-based interface

Speech-to-Text: OpenAI Whisper

Text-to-Speech: macOS say command + ffmpeg for audio conversion

PDF Processing: pypdf 3.17.0s

Environment Management: uv (Universal Virtualenv Manager)

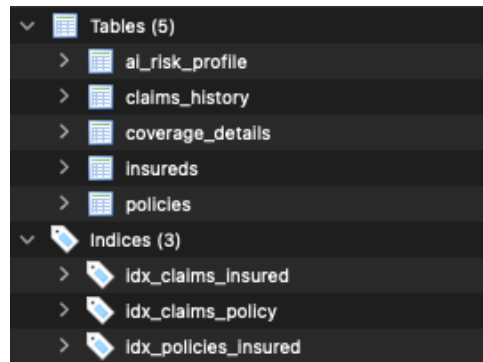
Note: This Agent was built, tested, and demoed on an Apple Mac Studio (M1 Max Chip, 10-core CPU, 32-core GPU, 64 GB RAM). All installation, configuration and demonstration steps provided in this document and accompanying videos and GitHub Repo are specific to this system architecture.

Dataset Details

The system utilizes two complementary datasets of fictional data for the *AI Agent Insure* company:

1. Structured SQLite Database

The SQLite database (insureds.db) contains 100 insured customers with comprehensive insurance data across five normalized tables:



ai_risk_profile (208 records): AI system footprint and underwriting risk factors (AI_System_Type, Risk_Score_Internal, Has_High_Risk_Use_Cases, Model_Complexity, Data_Sensitivity_Level)

claims_history (46 records): Historical AI incident claims (Claim_Number, Claim_Date, Claim_Type, Claim_Status, Claim_Amount, Description)

coverage_details (208 records): Coverage configuration for each policy (Coverage_Limit, Deductible, Coverage_Tier, Optional_AddOns_Selected, Regulatory_AddOns_Selected, Business_Interruption_Coverage, Cyber_Extension, Incident_Response_AddOn)

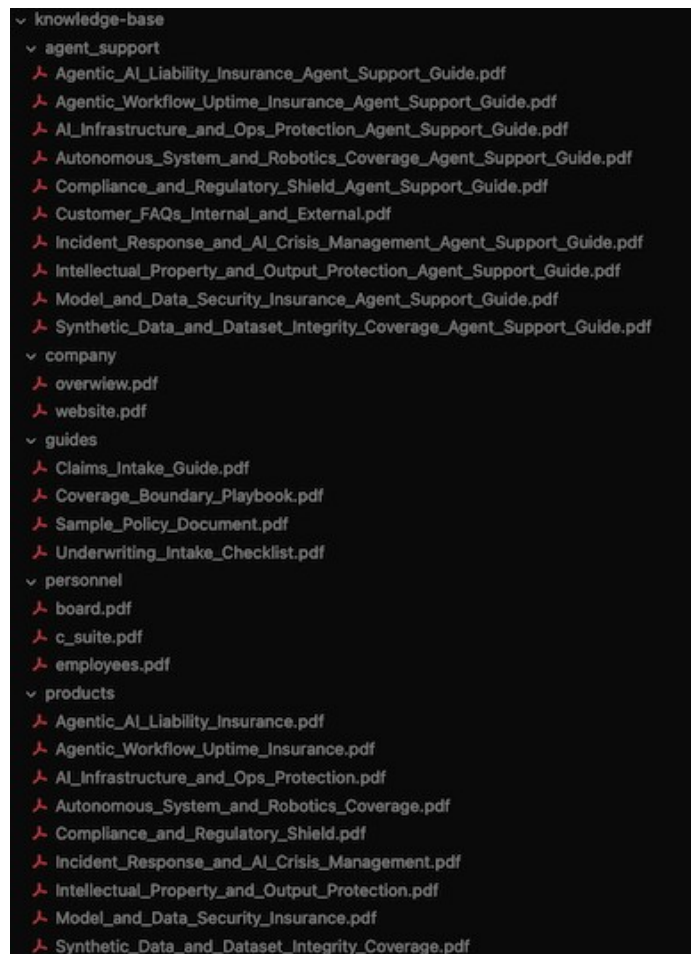
insureds (100 records): Core customer information including PII, demographic data, and business information (First_Name, Last_Name, Date_of_Birth, Phone_Number, Email_Address, Address, City, State, Postal_Code, Company_Name, Industry, Contact_Person)

policies (208 records): Master policy list with 1-3 policies per insured (Policy_Type, Effective_Date, Expiration_Date, Policy_Status, Payment_Status, Annual_Premium, Monthly_Premium, Payment_Method, Broker, Discounts_Applied)

Total Records: 770 records across all tables, linked by shared keys (Insured_ID, Policy_Number).

2. Unstructured Knowledge Base

An unstructured knowledge base of 25+ PDF documents organized into categories:



agent_support/: Agent support guides for various products (9 support guides)

company/: Company overview and website information

guides/: Process guides (claims intake, underwriting, coverage boundaries, sample policy documents)

personnel/: Organizational information (board, C-suite, employees)

products/: Product brochures and coverage details (9 product PDFs)

These PDFs are processed into vector embeddings using nomic-embed-text and stored in a ChromaDB vector store with enhanced metadata including document type, chunk indices, and page counts.

Software Installation / Configuration

The following software must be installed on your local machine (versions / steps in this document are Mac specific).

1. UV Package Manager

- Download and install the latest version [uv](#)

A terminal window titled 'rob@Roberts-Mac-Studio:~' showing the command 'uv --version' being executed. The output is 'uv 0.9.8 (85c5d3228 2025-11-07)'. There are green checkmarks in the right margin of the terminal window.

```
rob@Roberts-Mac-Studio:~  
uv --version  
uv 0.9.8 (85c5d3228 2025-11-07)
```

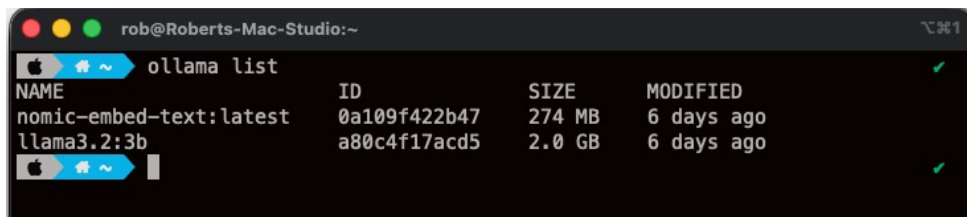
2. Ollama

- Download and install the latest version of [Ollama](#)
- Pull the [llama3.2:3b](#) and [nomic-embed-text](#) Models

ollama pull llama3.2:3b

ollama pull nomic-embed-text

ollama list

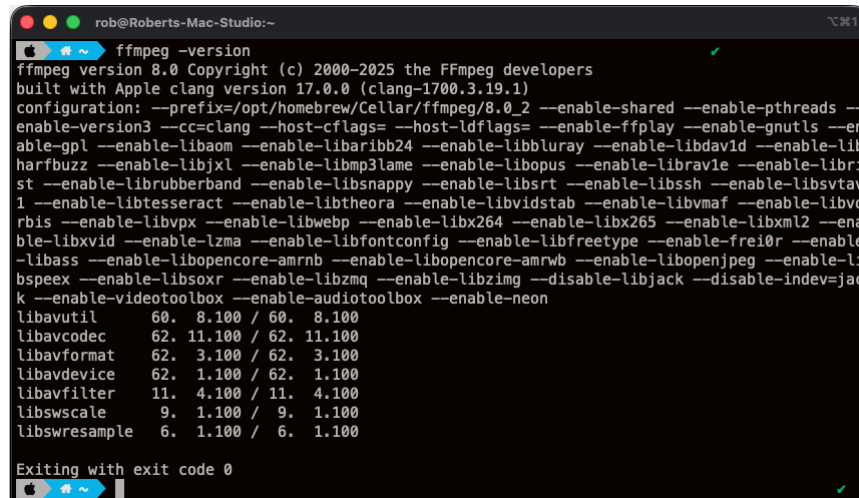
A terminal window titled 'rob@Roberts-Mac-Studio:~' showing the command 'ollama list' being executed. The output is a table with columns NAME, ID, SIZE, and MODIFIED. The table lists two models: 'nomic-embed-text:latest' and 'llama3.2:3b'. There are green checkmarks in the right margin of the terminal window.

```
rob@Roberts-Mac-Studio:~  
ollama list  
NAME                ID                SIZE    MODIFIED  
nomic-embed-text:latest 0a109f422b47    274 MB  6 days ago  
llama3.2:3b          a80c4f17acd5    2.0 GB  6 days ago
```

3. ffmpeg

- Download and install the latest version ffmpeg

brew install ffmpeg



```
rob@Roberts-Mac-Studio:~$ ffmpeg -version
ffmpeg version 8.0 Copyright (c) 2000-2025 the FFmpeg developers
built with Apple clang version 17.0.0 (clang-1700.3.19.1)
configuration: --prefix=/opt/homebrew/Cellar/ffmpeg/8.0_2 --enable-shared --enable-pthreads --
enable-version3 --cc=clang --host-cflags= --host-ldflags= --enable-ffplay --enable-gnutls --en
able-gpl --enable-libaom --enable-libaribb24 --enable-libbluray --enable-libdav1d --enable-lib
harfbuzz --enable-libjxl --enable-libmp3lame --enable-libopus --enable-librav1e --enable-libri
st --enable-librubberband --enable-lbsnappy --enable-lbsrt --enable-libssh --enable-lbsvtav
1 --enable-libtesseract --enable-libtheora --enable-libvidstab --enable-libvmaf --enable-libvo
rbis --enable-libvpx --enable-libwebp --enable-libx264 --enable-libx265 --enable-libxml2 --ena
ble-libxvid --enable-lzma --enable-libfontconfig --enable-libfreetype --enable-frei0r --enable
-libass --enable-libopencore-amrnb --enable-libopencore-amrwb --enable-libopenjpeg --enable-li
bspeex --enable-libsoxr --enable-libzmq --enable-libzimg --disable-libjack --disable-indev=jac
k --enable-videotoolbox --enable-audiotoolbox --enable-neon
libavutil      60. 8.100 / 60. 8.100
libavcodec     62. 11.100 / 62. 11.100
libavformat    62. 3.100 / 62. 3.100
libavdevice    62. 1.100 / 62. 1.100
libavfilter    11. 4.100 / 11. 4.100
libswscale     9. 1.100 / 9. 1.100
libswresample  6. 1.100 / 6. 1.100

Exiting with exit code 0
```

Application Startup Sequence

The application performs the following initialization steps:

1. **Load Whisper Model:** Loads OpenAI Whisper model for speech-to-text
2. **Connect to SQL Database:** Connects to `../insured-db/insureds.db` and displays statistics
3. **Load Vector Store:** Loads pre-created ChromaDB from `../vector-store/chroma_db/`
4. **Setup RAG Chain:** Initializes LLM, memory, and retrieval chain with optimized settings
5. **Run Test Query** (optional): Executes a test query to verify system functionality
6. **Launch Gradio Interface:** Starts web interface at <http://127.0.0.1:7861>

Steps to Launch Agent

Perform the following steps to launch the Agent after completing the steps above:

1. Clone [Repo](#)
2. Copy `sample.env.txt` file in root of project and save as `.env`
3. Open a Terminal and execute the following (ensure LLM and embedding models are pulled):

```
ollama serve
```

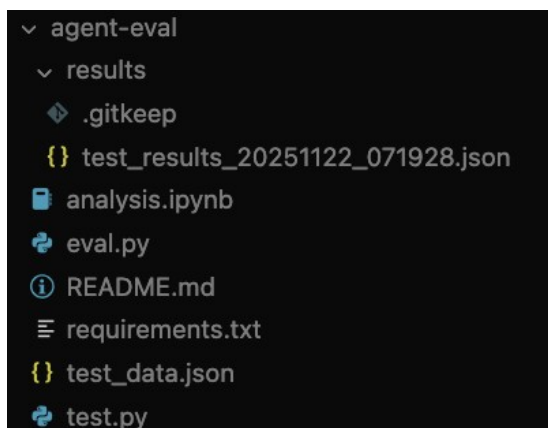
4. Open another Terminal, in the root of the project structure, and execute:

```
./init.sh
```

5. Open a Browser and load <http://127.0.0.1:7861/> to launch the Agent UI

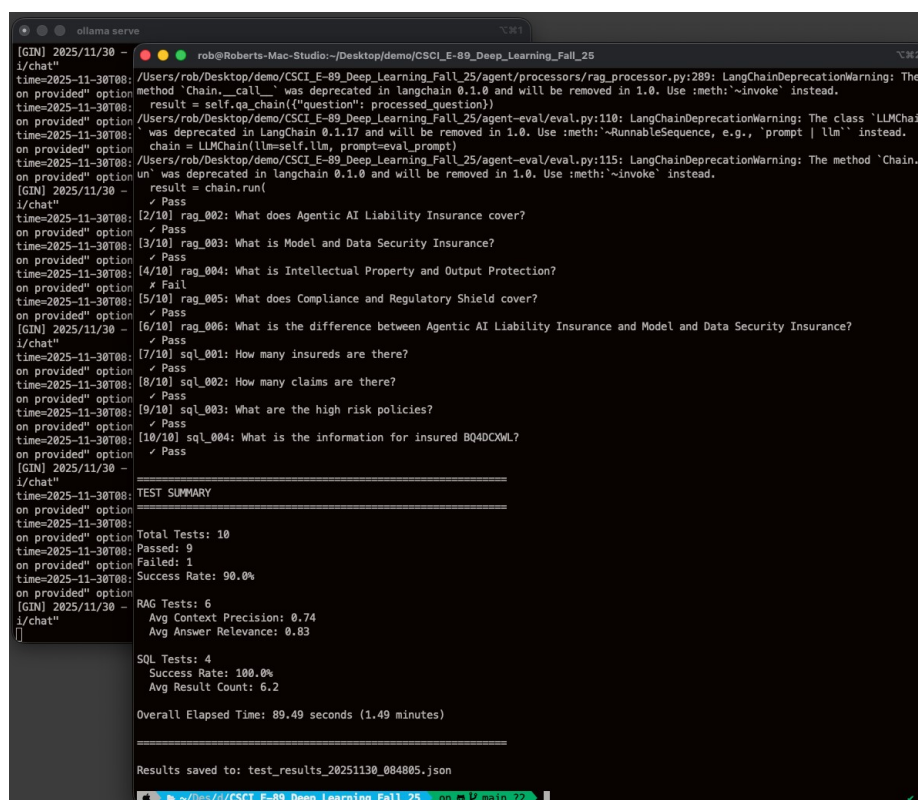
Agent Evaluation App

In addition to the local, hybrid RAG Agent, I created an Agent Evaluation App as part of this project. The code for this App is located in the agent-eval folder in the project structure.



To launch the App:

- Ensure the UV env manager is installed on your system.
- Ensure Ollama is running and the llama3.2:3b and nomic-embed-text models have been pulled.
- Launch a terminal, in the root of the project, and execute: *./init.sh*
- Then execute: *python agent-eval/test.py*



After completing the above steps, in the same terminal, execute the following to view the test results in a Jupyter Notebook: `uv run --with jupyter jupyter lab./agent-eval/analysis.ipynb`

```
[5]: # Show summary metrics
agg = data['aggregate']

print("\nTEST RESULTS")
print("=" * 50)
print(f"Success Rate: {agg['success_rate']:.1%}")
print(f"Passed: {agg['successful_tests']}/{agg['total_tests']}")

if agg.get('rag_tests', 0) > 0:
    print(f"\nRAG Tests: {agg['rag_tests']}")
    print(f"    Avg Context Precision: {agg.get('rag_avg_context_precision', 0):.2f}")
    print(f"    Avg Answer Relevance: {agg.get('rag_avg_answer_relevance', 0):.2f}")

if agg.get('sql_tests', 0) > 0:
    print(f"\nSQL Tests: {agg['sql_tests']}")
    print(f"    Success Rate: {agg.get('sql_success_rate', 0):.1%}")
```

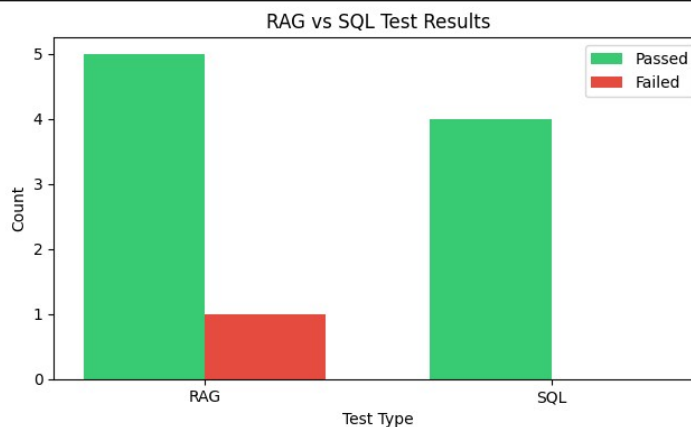
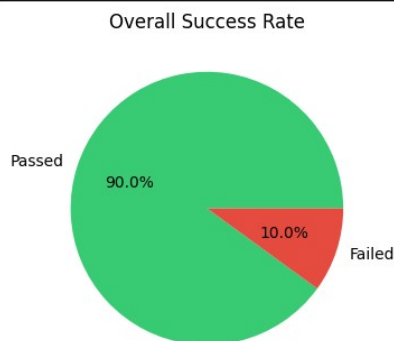
TEST RESULTS

=====

Success Rate: 90.0%
Passed: 9/10

RAG Tests: 6
Avg Context Precision: 0.74
Avg Answer Relevance: 0.83

SQL Tests: 4
Success Rate: 100.0%

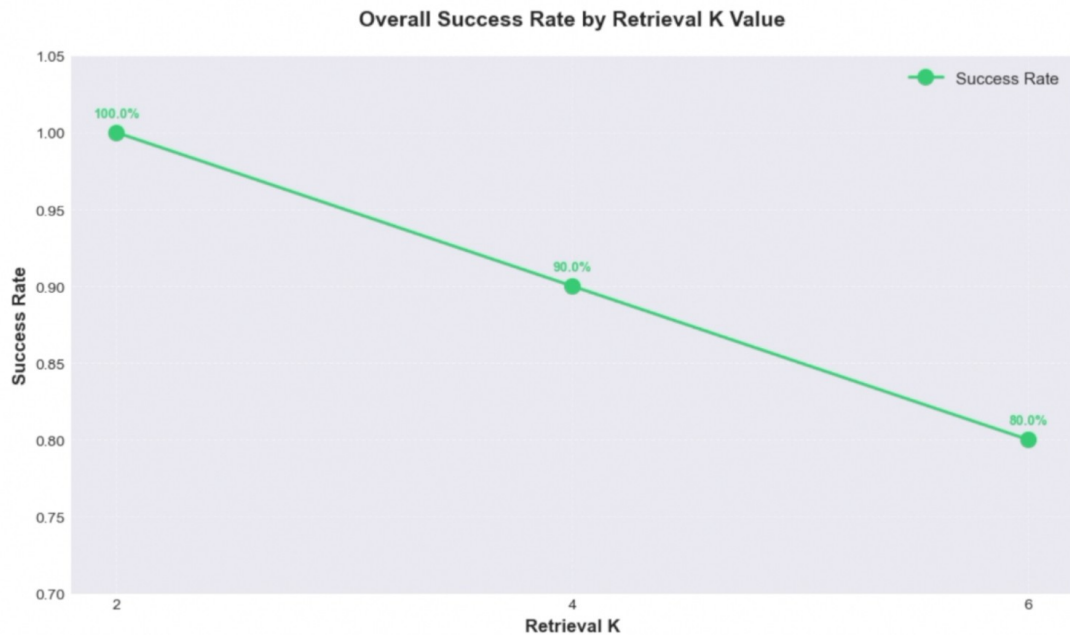


Lessons Learned

One of the more surprising findings while building this project came when running the Agent Eval App after testing the Agent with Retrieval K values of 2, 4, and 6. Ordinarily, we would expect the higher retrieval values to outperform the lower setting. However, this was not the case in these tests.

Overall Success Rate Comparison

```
In [6]: plot_metric(df, 'Retrieval_K', 'Success_Rate', '#2ecc71',  
                  'Overall Success Rate by Retrieval K Value', 'Success Rate',  
                  annotation_format='.1%', y_lim=(0.7, 1.05),  
                  legend_label='Success Rate')
```



This

may be caused by:

1. Inherent randomness in the Model's retrieval and generation processes
2. Specific documents retrieved between requests may differ based on vector similar calculations
3. The *Lost in the Middle* effect ([resource](#))

Resources

2 min (short) video: <https://youtu.be/HinayJJ687k>

15 min (long) video: <https://youtu.be/V0mAQjiUK6Q>

YouTube Project Playlist: <https://www.youtube.com/playlist?list=PLRBkbp6t5gM12JGRlbGomEETO2RknoRbP>

GitHub Repo: https://github.com/RobertFrenette/CSCI_E-89_Deep_Learning_Fall_25